

CS336 作业 3 (scaling): 缩放定律

版本 26.0.5

CS336 课程团队

2026 年春季

1 作业概述

在本次作业中，你将通过实践了解语言模型的缩放定律。

任务背景

你负责训练 ClosedAI 的下一代语言模型。训练这个模型将耗费相当于一个小国国内生产总值 (GDP) 的资金，还会融化一片南极冰架，因此你希望确保把这件事做好。具体而言，你有一笔固定的计算预算（以实际运行时间，即 wall-clock time 衡量）。你的目标是得到训练损失最低的模型，也就是一个计算最优的模型。为实现这一目标，你需要弄清楚如何在以下两者之间做出最佳权衡：(1) 训练更大的模型；(2) 使用更多 token 进行训练。

缩放定律通过经验关系，将语言模型的训练损失与模型规模、训练所用的计算量联系起来，常用于预测上述权衡关系 [J. Kaplan 等人, 2020; J. Hoffmann 等人, 2022]。在本次作业中，你将构建缩放定律，估计在 48 个 B200 小时的预算下训练时，计算最优的模型规模及其相应超参数。你不需要自己训练模型，而是向训练 API 提交：(1) 模型超参数（例如 Transformer 层数、嵌入维度、注意力头数、批大小、学习率、训练 token 数等）；(2) 该实验允许的最长实际运行时间。API 将返回使用相应超参数训练后得到的验证损失。用于拟合缩放定律的预算是额外的 12 个 B200 小时（相当于大型训练运行 FLOPs 预算的 25%）。训练 API 接受的超参数取值范围很广；为了有效利用缩放定律实验预算，你需要自行判断应当探索搜索空间中的哪些部分。

代码概览

本作业说明可在以下 GitHub 仓库中找到：

github.com/stanford-cs336/assignment3-scaling

请使用 Git 克隆该仓库。如有更新，我们会通知你；你可以运行 `git pull` 获取最新版本。

提交方式

你需要向 Gradescope 提交以下文件：

- `writeup.pdf`: 完整描述你拟合缩放定律的方法，以及如何利用该定律预测给定 FLOPs 预算下的最优模型规模。报告应足够详细，使他人能够复现你的结果。
- `code.zip`: 包含你为拟合缩放定律和计算估计值而编写的全部代码。

此外，还需通过 API 提交你预测的最优超参数和最终验证损失。

本次作业的部分成绩将由你预测的最优模型的性能决定。

关于 AI 工具的声明

AI 能够完全自主地完成作业中的许多部分。这会让你更难深入接触课程材料并从中学习。

允许使用 AI 工具回答高层次的概念性问题，或提供函数签名、库 API 等底层编程文档信息。但是，不得使用 AI 工具实现任何作业的任何部分。这既包括编程智能体（例如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（例如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它使用所提供的 AGENTS.md 文件。使用聊天机器人时，也应包含该提示词。

我们强烈建议你在完成作业时，关闭 IDE 中的 AI 自动补全功能（例如 Cursor Tab、GitHub Copilot）；非 AI 自动补全，例如自动补全函数名，则完全可以使用。往届学生特别提到，关闭 AI 自动补全后，更容易深入理解课程材料。

完整的 AI 使用政策请参阅[这份文档](#)。

2 缩放定律回顾

我们将回顾 Chinchilla 论文[J. Hoffmann 等人, 2022]中拟合缩放定律的一种方法。核心问题是：给定用于训练大型语言模型的计算预算 C ，选择什么样的超参数（模型规模、训练 token 数等），能够得到最低的训练损失？主要挑战在于如何把小规模实验的结果外推到更大规模。在本作业第二部分的实践中，你也可以借鉴其他参考文献中的思路，例如 J. Kaplan 等人 [1] 和 G. Yang 等人 [3]。

2.1 基于 IsoFLOPs 曲线的缩放定律

回顾一下：在包含 D 个 token 的数据集上，训练一个具有 N 个参数的 Transformer，所需的计算预算近似为 $C = 6ND$ 。J. Hoffmann 等人 [2] 中的 IsoFLOPs 缩放定律方法如下：对于每个计算预算 C ，在该预算内训练一系列不同规模 N 的语言模型（数据规模为 $D = \frac{C}{6N}$ ），得到最终训练损失 L 。

这样就会得到一组训练运行：它们使用的 FLOPs 均为 C_i ，但模型规模 N_{ij} 各不相同。J. Hoffmann 等人 [2] 通过实验观察到，在固定计算预算 C_i 下，最终训练损失 L_{ij} 与模型规模 N_{ij} 之间存在二次关系。对此的一种直观理解是：当 N_i 极小时，无论投入多少计算量，模型都无法拟合数据，因此这一范围内的最终训练损失很高。随着模型规模增大，最终训练损失平滑下降；但超过某个点后，模型就开始变得过大，以至于在 C FLOPs 内无法执行足够多次梯度更新来有效训练它。极端情况下，当 $N_i \rightarrow \infty$ 时，仅进行一次梯度更新就会超出计算预算 C ，训练因而会在损失仍很高时停止。寻找缩放定律的方法是：先确定每个 C_i 下的最优模型规模与数据集规模，再拟合一个根据目标 C 预测 N 和 D 的幂律；这个目标预算通常比此前训练运行所用的预算更大。

为拟合这些幂律，我们针对每个预算 C_i ，利用该预算下的一组训练运行（即其“IsoFLOPs 曲线”），找出使训练损失最小的模型规模 $N_{\text{opt}}(C_i)$ ，从而得到拟合所需的数据点。这个过程产生一系列数对 $\langle C_i, N_{\text{opt}}(C_i) \rangle$ ，并可类似地得到关于 D_{opt} 的数对。我们利用这些数据点拟合幂律 $N_{\text{opt}} \propto C^a$ 和 $D_{\text{opt}} \propto C^b$ ，再将其外推到目标计算预算，得到计算最优的模型规模与数据集规模。

题目 (chinchilla_isoflops): IsoFLOPs 缩放定律 (5 分)

编写一个脚本，使用一组训练运行的最终训练损失，复现上述用于拟合缩放定律的 IsoFLOPs 方法。本题使用 `data/isoflops_curves.json` 文件提供的训练运行数据（这些数据是合成的）。该文件包含一个 JSON 数组，其中每个元素都是一个描述单次训练运行的对象。下面给出前两次运行，以说明数据格式：

```
[
  {
    "parameters": 49999999,
    "compute_budget": 6e+18,
    "final_loss": 7.192784500319437
  },
  {
    "parameters": 78730505,
    "compute_budget": 6e+18,
    "final_loss": 6.750171320661809
  },
  ...
]
```

拟合缩放定律时，`scipy` 包可能会有所帮助，尤其是 `scipy.optimize.curve_fit`；不过，你可以采用任何你喜欢的曲线拟合方法。虽然 J. Hoffmann 等人 [2] 通过为每条 IsoFLOPs 曲线拟合二次函数来寻找最小值，但我们建议你直接将每个计算预算下训练损失最低的那次运行作为最小值。

- (a) 展示你外推得到的计算最优模型规模，以及你得到的 $\langle C_i, N_{\text{opt}}(C_i) \rangle$ 数据点。对于 10^{23} FLOPs 的预算，你预测的最优模型规模是多少？对于 10^{24} FLOPs 呢？

交付内容：一张展示模型规模随计算预算变化的缩放定律的图，标出用于拟合该定律的数据点，并至少外推到 10^{24} FLOPs。然后，用一句话给出你预测的最优模型规模。

- (b) 展示你外推得到的计算最优数据集规模，以及训练运行产生的 $\langle C_i, D_{\text{opt}}(C_i) \rangle$ 数据点。对于 10^{23} 和 10^{24} FLOPs 的预算，你预测的最优数据集规模分别是多少？

交付内容：一张展示数据集规模随计算预算变化的缩放定律的图，标出用于拟合该定律的数据点，并至少外推到 10^{24} FLOPs。然后，用一句话给出你预测的最优数据集规模。

3 构建缩放定律

本节中，你将在固定计算预算下，使用通过训练 API（第 3.2 节）查询得到的训练运行结果来拟合缩放定律。你的主要目标是选择模型规模和超参数，使模型在 48 个 B200 小时的预算下达到最低验证损失；同时，你还需要预测该模型将达到的验证损失。为了给预测的最优规模模型设置超参数，我们建议你分析在较小规模设置下，超参数如何影响验证损失。开始之前务必仔细规划各次运行：一旦超过用于缩放定律实验的 12 个 B200 小时预算，训练 API 就会拒绝后续请求。

3.1 训练运行详情

训练 API 会在 B200 GPU 上运行真实的训练任务。提交实验后，它会进入一个共享队列。处于排队中或运行中的实验，都会从你的 12 小时缩放定律预算中预留完整的 `max_runtime_seconds`。实验完成或失败后，API 会根据该实验报告的实际运行时间重新计算预算；该时间会被限制为至少 1 秒、至多 `max_runtime_seconds`。因此，如果你为一次运行预留了 30 分钟，而它在 12 分钟内完成，那么未使用的 18 分钟会返还至剩余预算。如果运行超时，则按 `max_runtime_seconds` 计费。

每个训练任务会运行 `n_evals` 个区段。API 在每个区段中训练 `total_train_tokens / n_evals` 个 token，每个区段结束后都在验证集上评估，并在 `status.val_losses` 中报告验证损失序列。对于已完成的实验，最终验证损失是该列表的最后一项。对于因超时而失败的实验，API 可能返回部分验证损失，但这些并不属于已完成的训练运行。

数据顺序是固定的。训练数据是经过分词的 DCLM 数据，最多包含 500B 个 token，不会对数据进行多轮遍历。对于每一种训练配置，API 都按相同的、确定性的训练 token 顺序读取数据。验证集同样使用 2^{18} 个验证 token。最终排行榜的验证损失将使用更多 token 计算。`model_seed` 字段只控制模型初始化，不控制数据顺序。

模型架构与作业 1 中的架构基本一致，但有少量差异。我们在作业仓库中提供了该模型的代码供你参考：<https://github.com/stanford-cs336/assignment3-scaling>。训练代码仅供参考。你应当只通过 API 训练模型。

模型在经过分词的 DCLM 数据上训练，词表大小为 32K，上下文长度为 512。API 支持 AdamW 和 SGD，两者均使用预热后余弦衰减的学习率调度。下方的 API 示例使用 AdamW，权重衰减为 0.01，梯度裁剪为 1.0，预热比例为 0.05，最终学习率比例为 0.1。训练使用长度为 512 个 token 的序列上的平均下一 token 交叉熵损失。API 模型不使用 dropout。

3.2 训练 API

你将使用这个训练 API，查询希望运行的缩放定律实验的最终验证损失。你的 API 密钥是你的 8 位斯坦福学号（例如 06123456）。请注意，必须连接斯坦福网络才能查询此 API，因此你可能需要使用 VPN。作为基本检查，你应当能够访问 API 文档页面 <http://hyperturing.stanford.edu:8000/docs>，并通过 `/budget` 端点验证 API 密钥是否有效（更多细节见下文）。训练 API 提供以下端点。

所有 API 请求都应在 `X-API-Key` 请求头中包含你的 API 密钥：

```
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> API_KEY = <YOUR_API_KEY_HERE>
>>> headers = {"X-API-Key": API_KEY}
>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}
```

你需要将实验提交至队列，轮询实验状态，并在实验完成后读取验证损失。

POST /submit

给定训练配置，将一次训练运行加入队列，并返回其实验 ID。请求体应为包含以下键的 JSON 对象：

- `architecture_config`: 描述 Transformer 架构的 JSON 对象。包含以下键：`attention_bias`、`head_dim`、`hidden_size`、`intermediate_size`、`num_attention_heads`、`num_hidden_layers`、`num_key_value_heads`、`rms_norm_eps`、`rope_theta`、`tie_word_embeddings`、`dtype` 和 `vocab_size`。`dtype` 字段必须为 `"float32"` 或 `"bfloat16"`。
- `optimizer_config`: 描述优化器的 JSON 对象。我们支持 AdamW 和 SGD 配置；下方示例使用 AdamW。
- `train_batch_size`: 整数形式的训练批大小。
- `val_batch_size`: 整数形式的验证批大小。
- `n_evals`: 训练期间执行验证评估的次数。
- `total_train_tokens`: 训练 token 的总数。
- `max_runtime_seconds`: 为本实验预留的最长实际运行时间。取值必须在 1 秒到 12 小时之间。
- `model_seed`: 用于初始化模型的随机种子。

API 将 `seq_len` 固定为 512，将 `n_validation_tokens` 固定为 $2^{18} \approx 262k$ ，并检查若干一致性条件。例如，`hidden_size` 必须等于 `num_attention_heads * head_dim`，`num_attention_heads` 必须能被 `num_key_value_heads` 整除，且 `total_train_tokens` 必须能被 `512 * train_batch_size` 整除。

响应是一个包含以下键的 JSON 对象：

- `experiment_id`: 已加入队列的实验的整数 ID。
- `budget_summary`: 包含 `used_seconds`、`remaining_seconds` 和 `total_budget_seconds` 的 JSON 对象。

如果你两次提交相同的训练配置，API 将返回 409 响应，并且不会额外预留预算。如果 `max_runtime_seconds` 超过剩余预算，API 将返回 400 响应。

要查询 API，请发送 HTTP POST 请求：

```
>>> import requests
>>> API_BASE_URL = "http://hyperturing.stanford.edu:8000"
>>> headers = {"X-API-Key": <YOUR_API_KEY_HERE>}
>>> training_config = {
...     "architecture_config": {
...         "attention_bias": False,
...         "head_dim": 64,
...         "hidden_size": 448,
...         "intermediate_size": 1280,
...         "num_attention_heads": 7,
...         "num_hidden_layers": 9,
...         "num_key_value_heads": 7,
...         "rms_norm_eps": 1e-6,
...         "rope_theta": 1_000_000,
...         "tie_word_embeddings": False,
...         "dtype": "bfloat16",
...         "vocab_size": 32_000,
...     },
... }
```

```

...     "optimizer_config": {
...         "lr_scheduler": {
...             "peak_value": 3e-4,
...             "final_lr_frac": 0.1,
...             "warmup_frac": 0.05,
...             "init_value": 0.0,
...         },
...         "weight_decay": 1e-2,
...         "beta1": 0.9,
...         "beta2": 0.95,
...         "eps": 1e-8,
...         "eps_root": 1e-8,
...         "grad_clip_norm": 1.0,
...     },
...     "train_batch_size": 128,
...     "val_batch_size": 32,
...     "n_evals": 16,
...     "total_train_tokens": 1_048_576,
...     "max_runtime_seconds": 30.0,
...     "model_seed": 0,
... }
>>> requests.post(f"{API_BASE_URL}/submit", headers=headers, json=training_config).json()
{'experiment_id': 1, 'budget_summary': {'used_seconds': 30.0, 'remaining_seconds': 43170.0,
'total_budget_seconds': 43200.0}}

```

GET /budget

给定 API 密钥，返回你的实验已预留或已使用的实际运行时间预算。响应是一个包含以下键的 JSON 对象：

- `used_seconds`：你提交的实验已使用或已预留的实际运行秒数总和。
- `remaining_seconds`：缩放定律预算中剩余的实际运行秒数。
- `total_budget_seconds`：缩放定律预算中的实际运行秒数总额。

要查询 API，请发送 HTTP GET 请求：

```

>>> requests.get(f"{API_BASE_URL}/budget", headers=headers).json()
{'used_seconds': 30.0, 'remaining_seconds': 43170.0, 'total_budget_seconds': 43200.0}

```

GET /experiments

返回使用你的 API 密钥提交的所有实验的列表。每个实验包含以下键：

- `experiment_id`：/submit 返回的实验 ID。
- `training_config`：为本实验提交的训练配置。
- `status`：实验的当前状态。

`status.status_type` 字段的取值为以下之一：

- `"queued"`：实验正在等待运行。
- `"running"`：实验正在运行。状态中包含 `val_losses`，即截至目前观察到的验证损失。实验在排队之后会处于 `running` 状态。该状态也包括启动时间和抢占时间，因此，实验显示为 `running` 的持续时间可能长于你指定的最大运行时间。

- "completed": 实验已结束。状态中包含 `used_runtime_seconds`、`val_losses` 和 `completed_at`。
- "failed": 实验失败或超时。因超时而失败的实验包含 `partial_val_losses`，你可以查看这些结果，但失败的实验均属于崩溃或超时的情况。

要查询 API，请发送 HTTP GET 请求：

```
>>> requests.get(f"{API_BASE_URL}/experiments", headers=headers).json()
[{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'queued',
'queued_at': '...'}}
```

GET /experiment/{experiment_id}

返回单个实验，其响应格式与 `/experiments` 返回的列表项相同。使用此端点轮询已提交的实验，直到其 `status.status_type` 为 "completed"：

```
>>> requests.get(f"{API_BASE_URL}/experiment/1", headers=headers).json()
{'experiment_id': 1, 'training_config': {...}, 'status': {'status_type': 'completed',
'val_losses': [3.1, 2.9, ...], ...}}
```

对于已完成的实验，使用 `status.val_losses` 的最后一项，作为该次训练运行的最终验证损失。

POST /final_submission

提交你预测的最优超参数和最终验证损失。请求体应包含：

- `training_config`: 你预测的最优训练配置。
- `predicted_final_loss`: 你预测该配置将达到的最终验证损失。

你可以重复向此端点提交；最新提交将替换上一次提交。

```
>>> final_submission = {
...     "training_config": training_config,
...     "predicted_final_loss": 2.75,
... }
>>> requests.post(f"{API_BASE_URL}/final_submission", headers=headers,
json=final_submission).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

GET /final_submission

返回你当前的最终提交；如果尚未提交，则返回 `None`：

```
>>> requests.get(f"{API_BASE_URL}/final_submission", headers=headers).json()
{'training_config': {...}, 'predicted_final_loss': 2.75, 'submitted_at': '...'}
```

3.3 排行榜

现在，你将使用训练 API，为 48 个 B200 小时的训练运行选择模型配置。目标是使这个最终模型的验证损失最小。

题目 (scaling_laws): 构建缩放定律排行榜 (50 分)

构建一个缩放定律，并用它选择模型规模和超参数，使你预计模型能够在 48 个 B200 小时的预算下达到最低验证损失。你还应预测该模型将达到的验证损失。为了构建缩放定律，你将使用我们的训练 API，查询各种实验配置下的最终验证损失（第 3.2 节）；用于拟合缩放定律的实验查询总量不得超过 12 个 B200 小时。这是 API 强制执行的硬性上限。

交付内容：一份经过排版的报告，完整描述你拟合缩放定律所采用的方案和方法。此外，报告还应说明你如何利用缩放定律预测给定 FLOPs 预算下的最优模型规模，并给出你的预测值。报告应解释为什么做出特定的设计决策，描述应足够详细，使他人能够复现你的方法和结果。

对于 48 个 B200 小时训练运行中你可以提交的超参数，我们基本不作限制。所有训练运行都在单张 B200 上进行。

为帮助你开始，我们建议至少思考以下问题。报告还应进一步说明，你针对下列每个因素是如何做出决策的：

- 在固定的 12 个 B200 小时缩放定律预算下，你如何决定查询哪些训练运行？
- 你如何拟合缩放定律？请描述你使用的具体方法，可能是一种或多种。特别是，熟悉 J. Kaplan 等人 [1] 和 J. Hoffmann 等人 [2] 所采用的方法，很可能会有所帮助。
- 你的缩放定律与实验数据的拟合程度如何？
- 对于我们给定的 48 个 B200 小时的 FLOPs 预算，你的缩放定律预测的最优模型规模是多少？预测损失是多少？
- 如果要训练一个参数量等于你预测的最优参数量的模型，你会使用哪些超参数？对于给定的模型超参数配置，可以用 $12n_{\text{layer}}d_{\text{model}}^2$ 估算非嵌入参数的数量。

除了报告，还需通过 API 提交：(1) 你预测的最优超参数；(2) 模型的验证损失。本次作业的部分成绩将由你预测的最优模型的性能决定。

参考文献

- [1] J. Kaplan 等人，《神经语言模型的缩放定律》(*Scaling Laws for Neural Language Models*)，2020 年。
- [2] J. Hoffmann 等人，《训练计算最优的大型语言模型》(*Training Compute-Optimal Large Language Models*)，2022 年。
- [3] G. Yang 等人，《张量程序 V：通过零样本超参数迁移调优大型神经网络》(*Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer*)，2022 年。